Hub RetailService — Project Documentation

Title Page

- **Project Name:** Hub RetailService – Technical Issue Reporting System
- **Subject:** Software Engineering / IT Project Management
- **Authors:** Szymon Kala, Mateusz Hofman
- **Version:** 1.0
- **Date:** May 30, 2026

Chapter 1: Business Description of the System

1.1 Introduction

Hub RetailService is a web-based helpdesk application designed specifically for retail store chains (supermarkets) and centralized distribution centers. The primary objective of the system is to streamline, accelerate, and automate the process of reporting and handling technical failures. By automating data collection and optimizing the workflow, the platform minimizes human involvement in administrative tasks, ensuring that issue tickets are immediately dispatched to the appropriate internal or external engineering departments.

1.2 Motivation for Software Development

In large retail networks, reporting equipment breakdowns, structural defects, or IT issues traditionally involves manual data entry, phone calls, or sending unmonitored emails to random departments. These operational inefficiencies lead to extended store downtime and data discrepancies in the database.

The main motivation behind Hub RetailService is the implementation of an **automatic facility identification mechanism**. First-line employees (e.g., cashiers or warehouse workers) do not need to know specific technical contacts or fill out long location forms when a failure is detected. They only need to enter the unique identification number of their store, and the system automatically retrieves full address details and assigns the appropriate regional service teams in the background.

1.3 Main Functions and Modules

The application architecture is logically divided into four independent modules:

- **Reporting Module (User View):** A simplified, lightweight web portal designed for quick issue reporting by store and warehouse staff.
- **Service Module (Technician Panel):** A specialized workspace where internal engineering departments (divided into IT, Buildings/Real Estate, and Security) manage their task queues. This module ensures strict data isolation so that departments do not overlap. After completing a repair, the technician can directly change the ticket status to "Resolved".

- **Administrative Module:** A command and configuration center for superusers, used to manage the global facility database, user permissions, and monitor the performance indicators of the entire system.
- **External Services Portal:** A dedicated panel for third-party outsourcing companies (external service providers), allowing them to accept, document, and execute advanced infrastructure repairs that require physical on-site intervention.

1.4 Target Audience and Stakeholders

Four main user roles are defined within the platform's ecosystem:

1. **Store and Warehouse Employees:** Personnel reporting ongoing failures directly from points of sale or logistics zones.
2. **Internal Technicians:** Domain experts (IT support, structural engineers, security managers) processing tickets within their isolated departments.
3. **System Administrators:** Individuals managing the global configuration, database structure, and monitoring process efficiency.
4. **External Service Providers (Vendors):** Third-party entities performing complex equipment or building infrastructure repairs.

1.5 Expected Business Benefits

- **Drastic Reduction in Reporting Time:** Immediate ticket generation thanks to automatic facility database lookup algorithms.
- **Elimination of Routing Errors:** System logic automatically and flawlessly allocates tasks to the correct technical teams.
- **Process Transparency:** Administrators can easily locate operational bottlenecks and monitor SLA compliance.
- **Data for Predictive Maintenance:** Collected technical logs allow analyzing which equipment types or regions generate the most failures.

Chapter 2: Software Requirements Specification (SRS)

2.1 Functional Requirements (FR)

System functionalities are defined using structural requirements and User Stories:

ID	Requirement Name	User Story / Description
FR-01	Automatic Data Retrieval	*As a store employee reporting a failure, I want the system to automatically fill in my market's name, address, and contact details after entering its number, so I can submit a ticket in seconds without manual errors.*
FR-02	Ticket Transfer Between Departments	*As an IT technician, I want to be able to seamlessly transfer a ticket to the Building department if a reported computer failure turns out to be caused by a burnt electrical outlet, so that the issue is rerouted without involving the store employee in a new submission.*
FR-03	Global Operational Overview	*As a system administrator, I want to have access to a dashboard showing all open tickets across all departments in real-time, so I can monitor work status across the entire retail network.*
FR-04	Status Updates and Logs	*As an external service technician, I need the ability to upload descriptions confirming the repair, so that my work can be verified and archived by the system.*
FR-05	Registration and Account Login	*As users, accounts are required to separate classes and access permissions.*
FR-06	Creation of Facilities / Categories	*The administrator creates facilities with numbers to be used by stores, as well as categories assigned to specific technicians.*

2.2 Non-Functional Requirements (NFR)

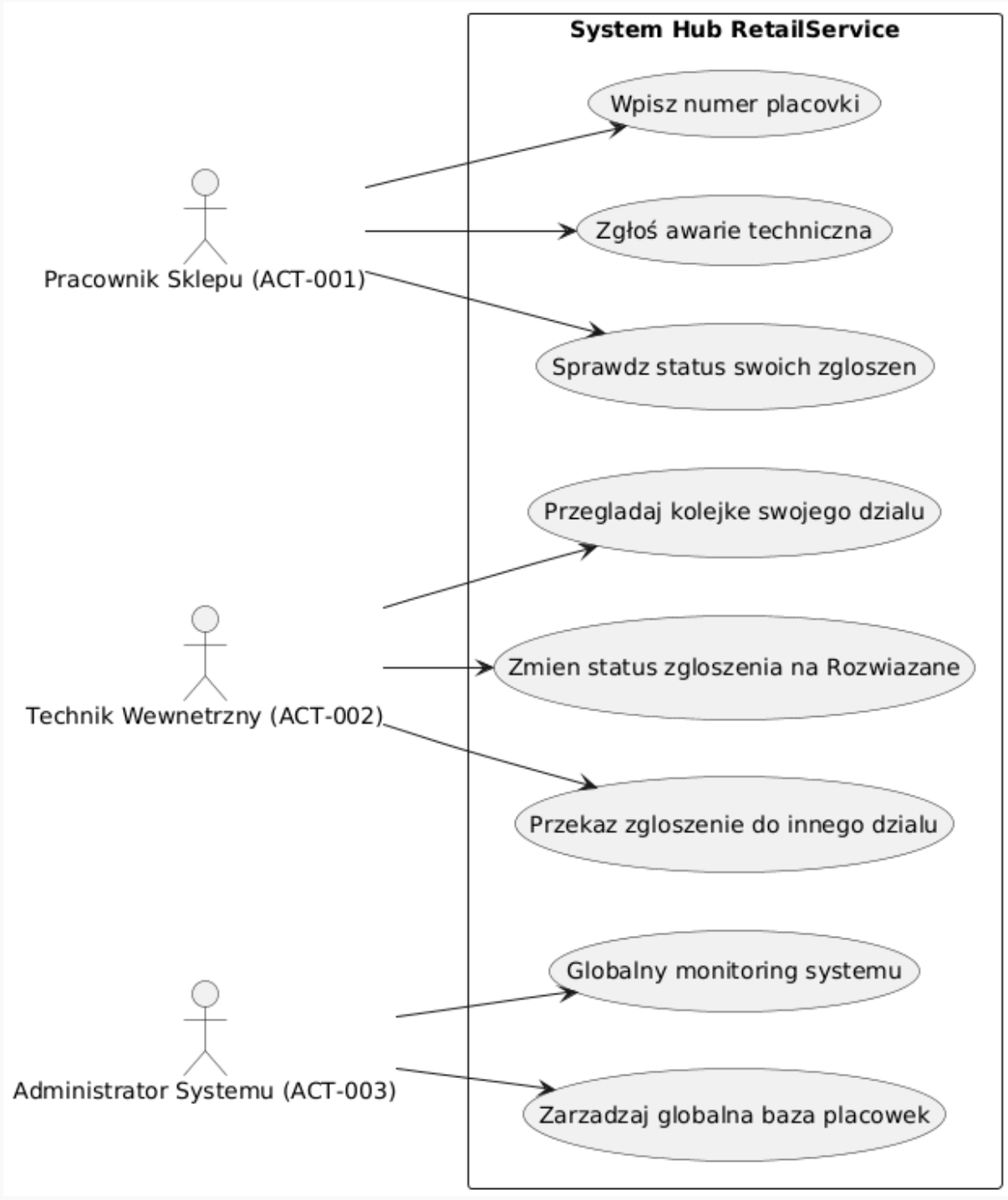
Technical parameters describing the system's operational environment:

- **NFR-01: Verification Performance:** The system must process the database query regarding the facility number and populate the form fields in less than 1.0 second.
- **NFR-02: Infrastructure Availability:** The server infrastructure must guarantee a minimum availability of 99.5% on a 24/7/365 basis to support the round-the-clock operations of supermarkets and warehouses.
- **NFR-03: Data Security and Privacy:** Sensitive employee data, manager phone numbers, and email addresses must be adequately protected in compliance with GDPR standards. Any attempt at mass database export by an administrator must generate an immutable security log.
- **NFR-04: Data Structure Isolation:** The database schema must enforce complete data separation; IT support users must not have the technical capability to view tickets assigned to the Security section.
- **NFR-05: Hardware and Browser Compatibility:** The responsive web interface must work flawlessly on older workstations in stores (older versions of Chrome/Edge) and on mobile Android data collectors.
- **NFR-06: Network Disconnection Resilience:** The ticket submission engine must be resilient to temporary internet outages; if a file attachment (photo) upload is interrupted, the form state and entered text must be cached and resumed once the network connection is restored.

Chapter 3: Models and Design of the Target System

3.1 Use Case Diagram

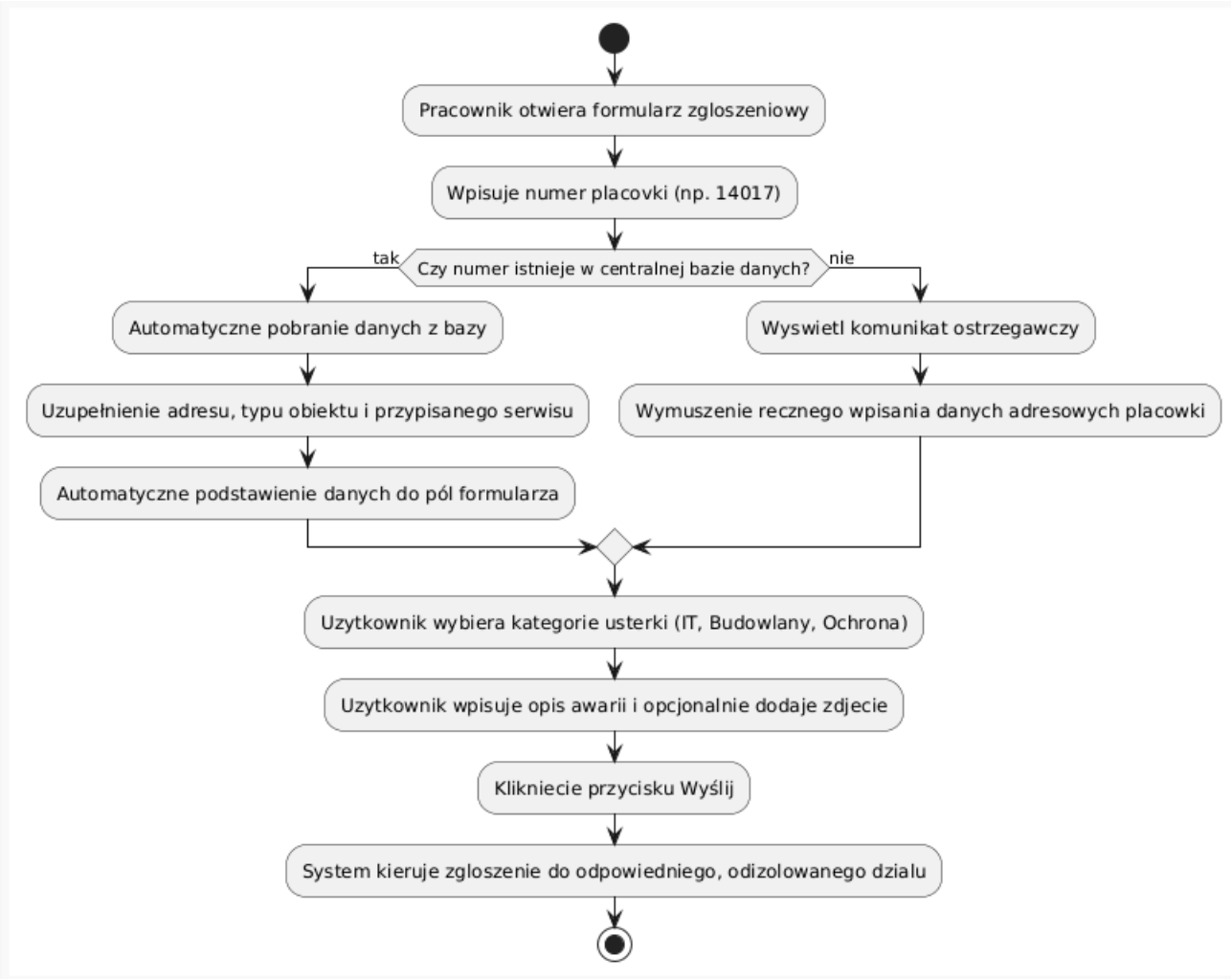
This diagram defines the system boundaries and illustrates how the three main internal actors interact with the system's use cases.



3.2 Activity Diagrams

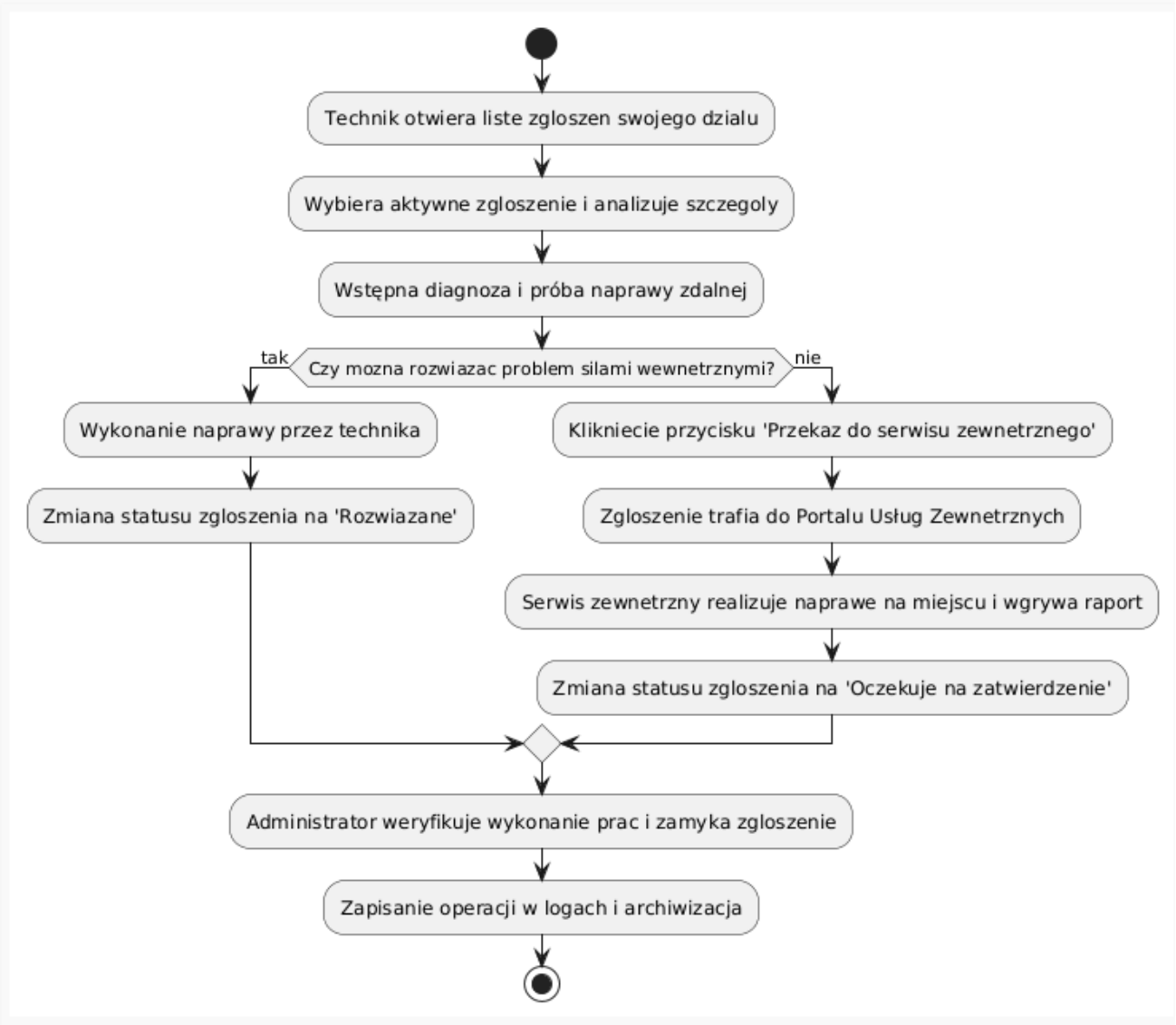
Aspect 1: Ticket Creation and Automatic Verification Process

This workflow shows the logic executed when a store employee enters a store number to automatically populate contact and address details.



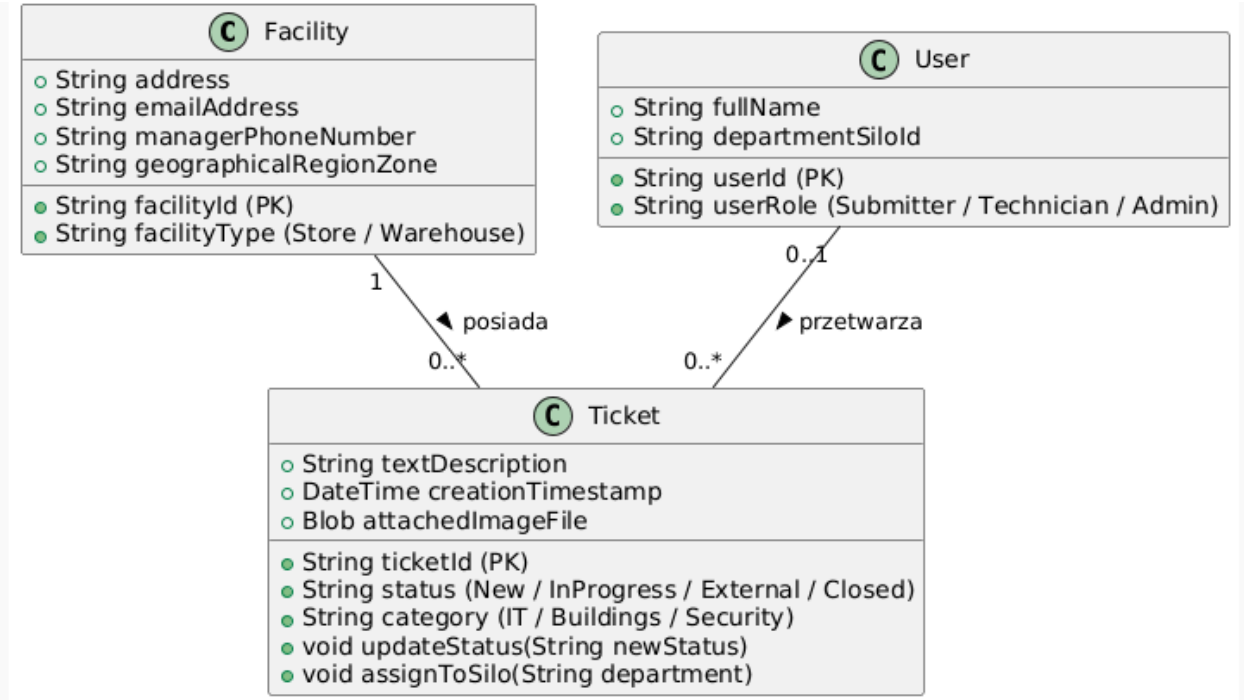
Aspect 2: Issue Verification, Escalation, and Closure Process

This diagram shows the path of a ticket from the moment it is picked up by an internal technician, including the critical decision point of internal vs. external repair.



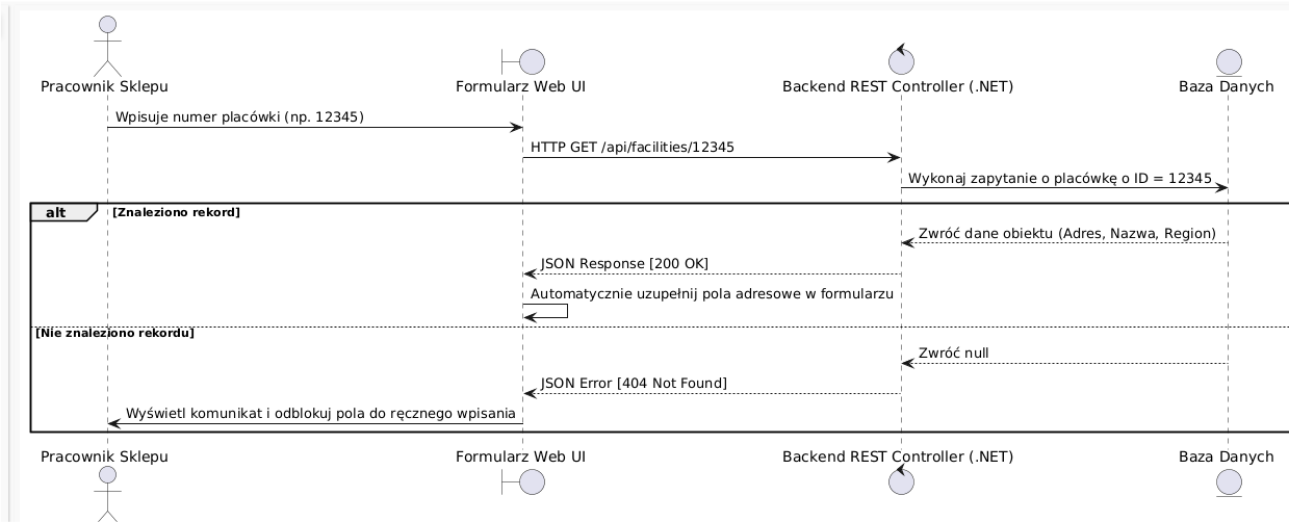
3.3 Class Diagram (Domain Model)

The data structure model reflecting the object relationships in the RetailService Hub database. The `Ticket` class is linked to a specific facility and the processing user.



3.4 Sequence Diagram

This diagram illustrates the real-time interaction between the Store Employee (User), the Browser (UI), the API Controller (.NET Core), and the Database during the automatic form population process (`FR-01`).



Chapter 4: System Prototype & Implementation

4.1 Technology Stack Description

The system prototype was designed and built based on modern architectural solutions within the Microsoft ecosystem, ensuring scalability and a clear separation of concerns in the codebase:

- **Architectural Pattern:** The application logic is based on the **Model-View-Controller (MVC)** pattern. It ensures the separation of business data (Models), the user interface

(Views), and the control layer (Controllers). This setup facilitates the implementation of requirement NFR-04 by enforcing security and data isolation rules directly at the routing controller level.

- **Backend Framework & Language:** The application is written in **C#** utilizing the **.NET Core** platform (ASP.NET Core). This framework provides high performance, asynchronous HTTP request handling, and built-in Dependency Injection (DI) containers.
- **Data Access Layer (ORM):** Communication with the database is mapped via **Entity Framework Core (EF Core)** using the Code-First / DbSet approach, allowing for seamless object-oriented operations within the C# code.
- **Database Tier:** The system utilizes a relational database (**Microsoft SQL Server** or **PostgreSQL**), optimized for rapid index lookups to fulfill requirement NFR-01.
- **Frontend Layer:** The user interface embedded within ASP.NET Core views (.cshtml) leverages HTML5, CSS3, JavaScript, and the **Bootstrap** framework, guaranteeing full interface responsiveness across desktop computers and mobile devices (NFR-05).